Finite Automata in Haskell

V. Velkey, W. Vromen

Tuesday 23rd May, 2023

Abstract

We give a toy example of a report in *literate programming* style. The main advantage of this is that source code and documentation can be written and presented next to each other. We use the listings package to typeset Haskell source code nicely.

Contents

1 How to use this?

To generate the PDF, open `report.tex` in your favorite LaTeXeditor and compile. Alternatively, you can manually do `pdflatex report; bibtex report; pdflatex report; pdflatex report` in a terminal.

You should have stack installed (see `https://haskellstack.org/`) and open a terminal in the same folder.

- To compile everything: `stack build`.
- To open ghci and play with your code: `stack ghci`
- To run the executable from Section 3: `stack build && stack exec myprogram`
- To run the tests from Section 4: `stack clean && stack test --coverage`

2 DFA Implementation

This describes our implementation of Deterministic Finite State Automata. As in the definition, a DFA consists of a tuple $(Q, \Sigma, \delta, q_{start}, A)$ representing the set of states, the alphabet, the transition function, the start state and the accept states.

```
module DFA where

type State = Int
type Symbol = Char
data DFA = DFA { states :: [[State]] -- For ease with subset construct, represent by set of
    states.
               , alphabet:: [Symbol]
               , delta :: Symbol -> [State] -> [State]
               ,  start:: [State]
               , acceptstate:: [[State]]}
```

We implement a basic evaluation function that upon input from the string, evaluates if the string is in the language.

```
evaluate:: DFA -> String -> Bool
evaluate df st = all (`elem` alphabet df) st && -- captures that all element of string in
    alphabet
                 stateArr (start df) st `elem` acceptstate df where
                     stateArr:: [State] -> String -> [State] -- recursively go to state at
                         end of string
                     stateArr q [] = q
                     stateArr q (x:xs) = stateArr (delta df x q) xs
```

We write a toy example DFA on the alphabet $\Sigma = \{0, 1\}$ computing the language of words starting with a 0.

```
zeroStart:: DFA
zeroStart = DFA [[0],[1],[2]] ['0', '1'] deltazero [0] [[1]] where
    deltazero:: Symbol -> [State] -> [State]
    deltazero char st
     | st == [0] && char == '0' = [1]
     | st == [0] && char == '1' = [2]
```

```
    | st == [1] = [1]
    | st == [2] = [2]
    | otherwise = [-1]
```

3 Wrapping it up in an exectuable

We will now use the library form Section 2 in a program.

```
module Main where

import DFA
import Data.List()
import Data.Maybe

main:: IO ()
main = undefined
```

The output of the program is something like this:

```
Hello!
[1,2,3,4,5,6,7,8,9,10]
[100,100,300,300,500,500,700,700,900,900]
[1,3,0,1,1,2,8,0,6,4]
[100,300,42,100,100,100,700,42,500,300]
GoodBye
```

Note that the above `showCode` block is only shown, but it gets ignored by the Haskell compiler.

4 Simple Tests

We now use the library QuickCheck to randomly generate input for our functions and test some properties.

```
module Main where

import Basics

import Test.Hspec
import Test.QuickCheck
```

The following uses the HSpec library to define different tests. Note that the first test is a specific test with fixed inputs. The second and third test use QuickCheck.

```
main :: IO ()
main = hspec $ do
  describe "Basics" $ do
    it "somenumbers should be the same as [1..10]" $
      somenumbers `shouldBe` [1..10]
    it "funnyfunction: result is within [1..100]" $
      property (\n -> funnyfunction n `elem` [1..100])
    it "myreverse: using it twice gives back the same list" $
      property $ \str -> myreverse (myreverse str) == (str::String)
```

To run the tests, use `stack test`.

To also find out which part of your program is actually used for these tests, run `stack clean && stack test`. Then look for "The coverage report for ... is available at ... .html" and open this file in your browser. See also: `https://wiki.haskell.org/Haskell_program_coverage`.

5 Optional: Profiling

The GHC compiler comes with a profiling system to keep track of which functions are executed how often and how much time and memory they take. To activate this RTS, we compile and execute our program as follows:

```
stack clean
stack build --profile
stack exec --profile myprogram -- +RTS -p
```

Results are saved in the file `myprogram.prof` which looks as follows. Note for example, that funnyfunction was called on 14 entries.

```
                                          individual      inherited
COST CENTRE       MODULE          no. entries  %time %alloc   %time %alloc

MAIN              MAIN            227        0    0.0    0.8     0.0  100.0
 main             Main            455        0    0.0   29.6     0.0   38.6
  funnyfunction   Basics          518       14    0.0    0.6     0.0    0.6
  randomnumbers   Basics          464        0    0.0    0.3     0.0    8.4
   randomRIO      System.Random   468        0    0.0    0.0     0.0    8.0
   ...
```

For many more RTS options, see the GHC documentation online at `https://downloads.haskell.org/~ghc/latest/docs/html/users_guide/profiling.html`.

6 Conclusion

Finally, we can see that [LW13] is a nice paper.

References

[LW13] Fenrong Liu and Yanjing Wang. Reasoning about agent types and the hardest logic puzzle ever. *Minds and Machines*, 23(1):123–161, 2013.